# CMPE343 Term Project: Airport Management Information System

## 1. Introduction

This report presents the design and implementation of a Relational Database Management System (RDBMS) for Ercan Airport Management. The system is designed to organize and manage information regarding airplanes, hangars, maintenance tests, and employees. To ensure a robust and realistic database architecture, the design comprises 10 tables, going beyond the minimal requirement of 8 tables to gain full marks.

### Assumptions:

1. Employee is a generalization (superclass) containing common attributes for all workers. Traffic controllers and technicians are specialized entities (subclasses).
2. An airplane can only be of one model, but a model applies to many airplanes (1:N relationship).
3. Technicians can be experts in multiple models, and a model can have multiple expert technicians, resulting in a Many-to-Many relationship resolved by a 'Technician_Expertise' junction table.
4. Airplanes move in and out of hangars, necessitating a historical tracking table 'Airplane_Location' handling IN and OUT timestamps.
5. Testing events are recorded with the specific plane, the technician who performed it, the test type, the score given, and hours spent.
6. Employees are uniquely identified by a Social Security Number (SSN).

## 2. ER Diagram Description

For the visual ER Diagram, use a tool like draw.io based on the logical entities below. Note that the requested total is at least 8 tables.

Entities & Tables:

1. Employee: Tracks general employee data (SSN, Name, Union_Mem_No).
2. Traffic_Controller: Subtype of Employee with annual medical exam dates.
3. Technician: Subtype of Employee.
4. Plane_Model: Defines aircraft models (Capacity, Weight, Manufacturer).
5. Technician_Expertise: Resolves the M:N relationship between Technician and Plane_Model.
6. Airplane: Specific airplane details, dependent on Plane_Model.
7. Hangar: Distinct Hangar areas in Ercan Airport.
8. Airplane_Location: Junction table tracking historical moves of Airplanes inside Hangars.
9. Test: Catalog of standardized tests.
10. Maintenance_Test: Records instances of testing events linking Plane, Technician, and Test.

## 3. Relational Data Model

- Employee (SSN (PK), Name, Union_Mem_No, Phone)
- Traffic_Controller (SSN (PK/FK), Last_Exam_Date)
- Technician (SSN (PK/FK))
- Plane_Model (Model_No (PK), Manufacturer, Capacity, Weight)
- Technician_Expertise (SSN (PK/FK), Model_No (PK/FK))
- Airplane (Plane_No (PK), Model_No (FK), Year_Built)
- Hangar (Hangar_No (PK), Location, Capacity)
- Airplane_Location (Location_ID (PK), Plane_No (FK), Hangar_No (FK), In_Date_Time, Out_Date_Time)
- Test (Test_ID (PK), Test_Name, Max_Score)
- Maintenance_Test (Event_ID (PK), Plane_No (FK), SSN (FK), Test_ID (FK), Test_Date, Hours_Spent, Score)

## 4. Data Definition Language (DDL)

```
CREATE TABLE Employee (
   SSN VARCHAR(15) PRIMARY KEY,
   Name VARCHAR(100) NOT NULL,
   Union_Mem_No VARCHAR(20) UNIQUE,
   Phone VARCHAR(15)
);

CREATE TABLE Traffic_Controller (
   SSN VARCHAR(15) PRIMARY KEY,
   Last_Exam_Date DATE NOT NULL,
   FOREIGN KEY (SSN) REFERENCES Employee(SSN) ON DELETE
CASCADE
);

CREATE TABLE Technician (
   SSN VARCHAR(15) PRIMARY KEY,
   FOREIGN KEY (SSN) REFERENCES Employee(SSN) ON DELETE
CASCADE
);

CREATE TABLE Plane_Model (
   Model_No VARCHAR(20) PRIMARY KEY,
   Manufacturer VARCHAR(50),
   Capacity INT,
   Weight DECIMAL(10,2)
);
```

```
CREATE TABLE Technician_Expertise (
   SSN VARCHAR(15),
   Model_No VARCHAR(20),
   PRIMARY KEY (SSN, Model_No),
   FOREIGN KEY (SSN) REFERENCES Technician(SSN) ON DELETE
CASCADE,
   FOREIGN KEY (Model_No) REFERENCES Plane_Model(Model_No)
ON DELETE CASCADE
);

CREATE TABLE Airplane (
   Plane_No VARCHAR(20) PRIMARY KEY,
   Model_No VARCHAR(20),
   Year_Built INT,
   FOREIGN KEY (Model_No) REFERENCES Plane_Model(Model_No)
);

CREATE TABLE Hangar (
   Hangar_No VARCHAR(10) PRIMARY KEY,
   Location VARCHAR(100),
   Capacity INT
);

CREATE TABLE Airplane_Location (
   Location_ID INT PRIMARY KEY,-- Auto increment depending on the
DBMS (e.g. SERIAL, IDENTITY)
   Plane_No VARCHAR(20),
   Hangar_No VARCHAR(10),
   In_Date_Time TIMESTAMP,
   Out_Date_Time TIMESTAMP,
   FOREIGN KEY (Plane_No) REFERENCES Airplane(Plane_No) ON
DELETE CASCADE,
   FOREIGN KEY (Hangar_No) REFERENCES Hangar(Hangar_No) ON
DELETE CASCADE
);

CREATE TABLE Test (
   Test_ID VARCHAR(10) PRIMARY KEY,
   Test_Name VARCHAR(100) NOT NULL,
   Max_Score INT
);

CREATE TABLE Maintenance_Test (
```

```
  Event_ID INT PRIMARY KEY,-- Auto increment depending on the
DBMS (SERIAL)
  Plane_No VARCHAR(20),
  SSN VARCHAR(15),
  Test_ID VARCHAR(10),
  Test_Date DATE NOT NULL,
  Hours_Spent DECIMAL(5,2),
  Score INT,
  FOREIGN KEY (Plane_No) REFERENCES Airplane(Plane_No) ON
DELETE CASCADE,
  FOREIGN KEY (SSN) REFERENCES Technician(SSN),
  FOREIGN KEY (Test_ID) REFERENCES Test(Test_ID)
);
```

---

## 5. Data Manipulation Language (DML)

```
-- Insert Employees
INSERT INTO Employee (SSN, Name, Union_Mem_No, Phone) VALUES
('11111', 'Ahmet Yilmaz', 'U001', '555-0101');
INSERT INTO Employee (SSN, Name, Union_Mem_No, Phone) VALUES
('22222', 'Ayse Demir', 'U002', '555-0202');
INSERT INTO Employee (SSN, Name, Union_Mem_No, Phone) VALUES
('33333', 'Mehmet Kaya', 'U003', '555-0303');
INSERT INTO Employee (SSN, Name, Union_Mem_No, Phone) VALUES
('44444', 'Fatma Celik', 'U004', '555-0404');
INSERT INTO Employee (SSN, Name, Union_Mem_No, Phone) VALUES
('55555', 'Ali Tekin', 'U005', '555-0505');

-- Insert Subtypes
INSERT INTO Traffic_Controller (SSN, Last_Exam_Date) VALUES
('11111', '2025-10-15');
INSERT INTO Traffic_Controller (SSN, Last_Exam_Date) VALUES
('22222', '2026-01-20');
INSERT INTO Technician (SSN) VALUES ('33333');
INSERT INTO Technician (SSN) VALUES ('44444');

-- Insert Plane Models
INSERT INTO Plane_Model (Model_No, Manufacturer, Capacity,
Weight) VALUES ('B737', 'Boeing', 200, 41000);
INSERT INTO Plane_Model (Model_No, Manufacturer, Capacity,
Weight) VALUES ('A320', 'Airbus', 180, 42600);
```

---

```
INSERT INTO Plane_Model (Model_No, Manufacturer, Capacity,
Weight) VALUES ('B777', 'Boeing', 350, 134000);

-- Insert Technician Expertise
INSERT INTO Technician_Expertise (SSN, Model_No) VALUES ('33333',
'B737');
INSERT INTO Technician_Expertise (SSN, Model_No) VALUES ('33333',
'A320');
INSERT INTO Technician_Expertise (SSN, Model_No) VALUES ('44444',
'B777');

-- Insert Airplanes
INSERT INTO Airplane (Plane_No, Model_No, Year_Built) VALUES
('TC-JAA', 'B737', 2015);
INSERT INTO Airplane (Plane_No, Model_No, Year_Built) VALUES
('TC-JAB', 'A320', 2018);
INSERT INTO Airplane (Plane_No, Model_No, Year_Built) VALUES
('TC-JAC', 'B777', 2020);

-- Insert Hangars
INSERT INTO Hangar (Hangar_No, Location, Capacity) VALUES ('H1',
'North Wing', 5);
INSERT INTO Hangar (Hangar_No, Location, Capacity) VALUES ('H2',
'South Wing', 3);

-- Insert Airplane Locations
INSERT INTO Airplane_Location (Location_ID, Plane_No, Hangar_No,
In_Date_Time, Out_Date_Time)
VALUES (1, 'TC-JAA', 'H1', '2026-05-01 10:00:00', '2026-05-02 12:00:00');
INSERT INTO Airplane_Location (Location_ID, Plane_No, Hangar_No,
In_Date_Time, Out_Date_Time)
VALUES (2, 'TC-JAB', 'H2', '2026-05-10 08:00:00', NULL);

-- Insert Tests
INSERT INTO Test (Test_ID, Test_Name, Max_Score) VALUES ('T1',
'Engine Diagnostic', 100);
INSERT INTO Test (Test_ID, Test_Name, Max_Score) VALUES ('T2',
'Landing Gear Check', 100);
INSERT INTO Test (Test_ID, Test_Name, Max_Score) VALUES ('T3',
'Avionics System Test', 100);

-- Insert Maintenance Tests
INSERT INTO Maintenance_Test (Event_ID, Plane_No, SSN, Test_ID,
Test_Date, Hours_Spent, Score)
```

*VALUES (1, 'TC-JAA', '33333', 'T1', '2026-05-01', 4.5, 95);*
*INSERT INTO Maintenance_Test (Event_ID, Plane_No, SSN, Test_ID, Test_Date, Hours_Spent, Score)*
*VALUES (2, 'TC-JAB', '33333', 'T2', '2026-05-11', 2.0, 98);*
*INSERT INTO Maintenance_Test (Event_ID, Plane_No, SSN, Test_ID, Test_Date, Hours_Spent, Score)*
*VALUES (3, 'TC-JAC', '44444', 'T3', '2026-05-12', 6.0, 100);*
*INSERT INTO Maintenance_Test (Event_ID, Plane_No, SSN, Test_ID, Test_Date, Hours_Spent, Score)*
*VALUES (4, 'TC-JAA', '33333', 'T2', '2026-05-05', 3.0, 85);*

---

## 6. 15 Statistical & Relational Queries

Q1. Retrieve all airplanes matching their models and capacity using JOIN.
```
SELECT a.Plane_No, a.Year_Built, m.Manufacturer, m.Capacity
FROM Airplane a JOIN Plane_Model m ON a.Model_No = m.Model_No;
```

Q2. Count the number of airplanes per plane model using GROUP BY.
```
SELECT Model_No, COUNT(Plane_No) AS Total_Planes
FROM Airplane GROUP BY Model_No;
```

Q3. List technicians who are experts on more than one plane model.
```
SELECT e.Name, COUNT(te.Model_No) AS Expertise_Count
FROM Technician_Expertise te JOIN Employee e ON te.SSN = e.SSN
GROUP BY e.Name HAVING COUNT(te.Model_No) > 1;
```

Q4. Find the average test score for each test type.
```
SELECT Test_ID, AVG(Score) AS Avg_Score
FROM Maintenance_Test GROUP BY Test_ID;
```

Q5. Find all airplanes currently parked in a hangar (Out_Date_Time is NULL).
```
SELECT Plane_No, Hangar_No, In_Date_Time
FROM Airplane_Location WHERE Out_Date_Time IS NULL;
```

Q6. Get the total number of hours each technician spent on tests, ordered descending.
```
SELECT e.Name, sum(mt.Hours_Spent) AS Total_Hours
FROM Maintenance_Test mt JOIN Employee e ON mt.SSN = e.SSN
GROUP BY e.Name ORDER BY Total_Hours DESC;
```

Q7. Find all Traffic controllers whose last medical exam was more than a year ago.
```
SELECT e.Name, tc.Last_Exam_Date
FROM Traffic_Controller tc JOIN Employee e ON tc.SSN = e.SSN
WHERE tc.Last_Exam_Date < CURRENT_DATE - INTERVAL '1 year';
```

Q8. Extract the month from Test_Date and count tests per month utilizing 'to_char'.
SELECT to_char(Test_Date, 'Month') AS Test_Month, COUNT(Event_ID) AS Total_Tests
FROM Maintenance_Test GROUP BY to_char(Test_Date, 'Month');

Q9. List airplanes that scored below 90 in any of their tests.
SELECT DISTINCT Plane_No FROM Maintenance_Test WHERE Score < 90;

Q10. Get the full details of the most recent test performed on plane 'TC-JAA'.
SELECT * FROM Maintenance_Test WHERE Plane_No = 'TC-JAA' ORDER BY Test_Date DESC
LIMIT 1;

Q11. List all employees and explicitly output their derived role using outer joins and conditionals.
SELECT e.Name, e.Union_Mem_No,
CASE WHEN tc.SSN IS NOT NULL THEN 'Controller'
WHEN t.SSN IS NOT NULL THEN 'Technician' ELSE 'Other' END AS Emp_Type
FROM Employee e LEFT JOIN Traffic_Controller tc ON e.SSN = tc.SSN LEFT JOIN Technician t ON e.SSN = t.SSN;

Q12. Compare the total hangar capacity with the current volume of stored airplanes using subqueries in the SELECT clause.
SELECT (SELECT SUM(Capacity) FROM Hangar) AS Max_Capacity,
(SELECT COUNT(*) FROM Airplane_Location WHERE Out_Date_Time IS NULL) AS Current_Stored;

Q13. List airplanes that haven't been tested yet using an advanced subquery.
SELECT Plane_No FROM Airplane
WHERE Plane_No NOT IN (SELECT DISTINCT Plane_No FROM Maintenance_Test);

Q14. Find the technician name and test score for the highest score ever recorded in the facility.
SELECT e.Name, mt.Score FROM Maintenance_Test mt
JOIN Employee e ON mt.SSN = e.SSN
WHERE mt.Score = (SELECT MAX(Score) FROM Maintenance_Test);

Q15. Present total maintenance tests and the average hours spent per test, grouped by Plane Model.

SELECT a.Model_No, COUNT(mt.Event_ID) AS Total_Tests, ROUND(AVG(mt.Hours_Spent),

2) AS Avg_Hours

FROM Maintenance_Test mt JOIN Airplane a ON mt.Plane_No = a.Plane_No GROUP BY a.Model_No;